7. AIM : To write a program to find ε – closure of all states of any given NFA with ε transition.

1. Start.

2. Read the number of alphabets and the alphabets of the NFA. Use `e` to represent the epsilon (ε) transition.

3. Read the number of states and the number of transitions in the NFA.

4. Read and store each transition in the form of source state, input symbol and destination state.

5. For each state of the NFA, initialize its ε-closure by adding the state itself.

6. Consider each state currently present in the ε-closure and check all the transitions from that state.

7. If a transition has `e` as its input symbol, identify its destination state and add it to the ε-closure if it is not already present.

8. Continue checking the newly added states and their ε-transitions until no new state can be added to the ε-closure.

9. Repeat the above process for every state of the NFA to obtain the ε-closure of all states.

10. Display the ε-closure of each state in the required format.

11. Stop.

8. AIM : To write a program to convert NFA to DFA.

1. **Start.**
2. Read the number of alphabets and store the alphabets.
3. Read the number of states, start state, and final states of the NFA.
4. Read the NFA transitions in the form:
   source state, input symbol, destination state.
5. Store the transitions in a transition table.
6. Create the initial DFA state as the set containing only the **NFA start state**.
7. Add the initial DFA state to the list of DFA states.
8. Take each unprocessed DFA state one by one.
9. For each input alphabet:
    - Check every NFA state present in the current DFA state.
    - Find all states reachable from those states using the given input symbol.
    - Combine all the reachable states to form a new DFA state.
10. If the newly obtained DFA state does not already exist, add it to the list of DFA states.
11. Repeat Steps 8–10 until all DFA states have been processed.
12. Check each DFA state:
    - If it contains at least one NFA final state, mark the DFA state as a **final state**.

13. Display all DFA states and their final-state status.
14. Display the start state of the DFA.
15. **Stop.**

9. AIM : To write a program to find First and Follow of any given grammar.

1. **Start.**
2. Read the number of productions and store all the productions of the grammar.
3. Use uppercase letters to represent non-terminals and other characters to represent terminals. Use `e` to represent epsilon (ε).
4. To find **FIRST** of a symbol:
    - If the symbol is a terminal, add the symbol to its FIRST set.
    - If the symbol is a non-terminal, find all productions having that non-terminal on the left-hand side.
    - If a production derives `e`, add `e` to the FIRST set.
    - Otherwise, find the FIRST of the symbols on the right-hand side from left to right.
    - Add the FIRST symbols except `e`. If a symbol can derive `e`, continue with the next symbol.
    - If all symbols can derive `e`, add `e` to the FIRST set.
5. To find **FOLLOW** of a non-terminal:
    - Add `$` to the FOLLOW set of the start symbol.
    - Search every production for the given non-terminal.
    - If another symbol occurs after it, find the FIRST of that symbol and add it to the FOLLOW set, excluding `e`.
    - If the following symbol can derive `e`, add the FOLLOW of the left-hand-side non-terminal.
    - If the non-terminal occurs at the end of a production, add the FOLLOW of the left-hand-side non-terminal.
6. Repeat the above process for all non-terminals in the grammar.
7. Display the FIRST set of each non-terminal.
8. Display the FOLLOW set of each non-terminal.
9. **Stop.**

10. AIM : To Design and implement a recursive descent parser for a given grammar.

This algorithm uses recursive procedures to implement the following grammar:

E → T E′
E′ → + T E′ | ε
T → F T′
T′ → * F T′ | ε
F → (E) | id

1. **Start.**
2. Read the arithmetic expression as input.
3. Initialize the input position `i` to `0` and `error` to `0`.
4. Call the function `E()` to start parsing.
5. In `E()`:
    - Call `T()`.
    - If the next symbol is `+`, move to the next symbol and call `T()` repeatedly.
6. In `T()`:
    - Call `F()`.
    - If the next symbol is `*`, move to the next symbol and call `F()` repeatedly.
7. In `F()`:
    - If the symbol is `a`, move to the next symbol.
    - If the symbol is `(`, move to the next symbol, call `E()`, and check for the matching `)`.
    - Otherwise, set `error = 1`.
8. After parsing, check whether the entire input has been processed and no error has occurred.
9. If `i` reaches the end of the input and `error = 0`, display **Accepted**.
10. Otherwise, display **Rejected**.
11. **Stop.**

11. AIM : To Construct a Shift Reduce Parser for a given language.

1. **Start.**
2. Define the operator precedence table for the symbols `i`, `+`, `*`, and `$`, where:
    - `<` indicates **Shift**.
    - `>` indicates **Reduce**.
    - `=` indicates equal precedence.
3. Read the input expression and ensure that it ends with the end marker `$`.
4. Initialize an empty stack and push `$` onto the stack.
5. Set the input pointer to the first symbol of the expression.
6. Compare the **top symbol of the stack** with the **current input symbol** using the precedence table.
7. If the relation is `<` or `=`, perform a **Shift** operation:
    - Push the current input symbol onto the stack.
    - Move the input pointer to the next symbol.
8. If the relation is `>`, perform a **Reduce** operation:
    - Pop the top symbol from the stack.
    - Continue popping symbols until a `<` relation is found between the stack symbol and the removed symbol.
    - Treat the removed symbols as a handle and complete the reduction.
9. Display the stack, remaining input, and the operation performed after each step.
10. Repeat the shift and reduce operations until the stack contains `$` and the current input symbol is also `$`.

11. If both are `$`, the expression is successfully parsed. Display **Valid Expression**.
12. If an invalid symbol or precedence relation is encountered, display **Invalid Expression**.
13. **Stop.**

12. AIM : To Write a program to perform constant propagation.

1. **Start.**
2. Read the number of intermediate code expressions.
3. Read each expression in the form of **operator, operand1, operand2, and result**.
4. Initialize the flag of each expression to indicate that it has not been processed.
5. Examine each expression one by one.
    - Check whether the operands are constants.
    - If both operands are constants, perform the corresponding operation (`+`, `-`, `*`, or `/`).
    - If the operator is `=`, take the operand value as the result.
6. Store the calculated constant value as a string and mark the expression as processed.
7. Search the subsequent expressions for the result variable of the processed expression.
8. Replace every occurrence of that variable in the operands with its constant value.
9. Continue the process for all the expressions in the intermediate code.
10. Display the expressions that are not eliminated after constant propagation.
11. **Stop.**

13. AIM : To Implement Intermediate code generation for simple expressions.

1. **Start.**
2. Read the arithmetic expression in infix form.
3. Convert the infix expression into **postfix form** using a stack and operator precedence.
4. Scan the postfix expression from left to right.
5. If the current symbol is an operand, push it onto the stack.
6. If the current symbol is an operator:
    - Pop the second operand from the stack.
    - Pop the first operand from the stack.
    - Create a new temporary variable such as `t1`.
    - Generate the three-address instruction:
      t1 = operand1 operator operand2
    - Push the temporary variable back onto the stack.
7. Repeat the process until the complete postfix expression is processed.
8. Display the generated **intermediate code (three-address code)**.
9. **Stop.**

14. AIM : To Implement the back end of the compiler which takes the three address code and produces the 8086 assembly language instructions that can be assembled and run using an 8086 assembler. The target assembly instructions can be simple move, add, sub, jump etc.

1. **Start.**
2. Read the number of **Three Address Code (TAC)** instructions.
3. Read and store each TAC instruction containing the operator, operands, and result.
4. Initialize the required variables and registers.
5. Select each TAC instruction one by one for code generation.
6. Identify the operator present in the instruction.
7. Load the first operand into the `AX` register using `MOV`.
8. Load the second operand into `BX` when required.
9. Generate the appropriate instruction such as `ADD`, `SUB`, `MUL`, or `DIV`.
10. Move the computed result from the register to the destination variable.
11. Repeat steps 5–10 for all TAC instructions.
12. Display the generated **8086 assembly language code**.
13. **Stop.**